

Javascript

class and object , inheritance, method overriding

Theory

1. Classes and Objects

- **Class** → A blueprint for creating objects. Defines properties and methods.
- **Object** → An instance of a class. Created using new.
- **Constructor** → Special method automatically called when an object is created.

Example:

```
class Student {  
  constructor(name, fees) {  
    this.name = name;  
    this.fees = fees;  
  }  
}  
  
let obj = new Student("Raj", 30000);
```

2. Inheritance

- **Definition:** Mechanism where one class (child) derives properties and methods from another (parent).
- **Keyword:** extends.
- **Benefit:** Code reuse and hierarchical relationships.

Example:

```
class Parent {
```

```
    greet() { console.log("Hello from Parent"); }  
}  
class Child extends Parent {  
    display() { console.log("Hello from Child"); }  
}
```

3. Method Overriding

- **Definition:** Subclass provides a new implementation of a method defined in the parent.
- **Keyword:** super → calls parent's method.

Example:

```
class Animal {  
    sound() { console.log("Generic sound"); }  
}  
class Dog extends Animal {  
    sound() {  
        super.sound();  
        console.log("Barking...");  
    }  
}
```

4. Key Concepts

- **Encapsulation** → Bundling data and methods together.
- **Polymorphism** → Same method name, different implementations.
- **Abstraction** → Hiding implementation details, exposing only essentials.

Coding Round Questions

- Class basics → Explain difference between class and object with example.
- Constructor → Show how constructors work in JavaScript classes.
- Inheritance → Write a program where a child class inherits from a parent class.
- Method overriding → Demonstrate overriding a parent method in a child class.
- Encapsulation → Explain encapsulation with a class example.
- Polymorphism → Show polymorphism using method overriding.

⚡ Machine Coding Round Challenge

Problem: Employee Management System

Requirements:

- Create a Person class with properties name, age.
- Create an Employee class extending Person with additional property salary.
- Override a method display() to show employee details.
- Create a Manager class extending Employee with property department.
- Demonstrate inheritance, constructor usage, and method overriding.

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Class & Object</title>
  </head>
  <body>
    <h2>Class and object</h2>

    <script>
      // class => blueprint for Object (maximum 1 bytes)
```

```

// object => instance of class

class Student {

    name;
    fees;

    constructor(name, fees){
        this.name = name;
        this.fees = fees;
        console.log("constructor is called");
    }

    greet(){
        console.log("greet method");
    }

    display() {
        console.log("Display method");
        this.greet();
    }
}

let obj = new Student("Redmi", 30000);

obj.display();

console.log(obj.name , obj.fees);

let obj1 = new Student("FIIT", 40000);

console.log(obj1.name , obj1.fees);


//   constructor => it's a special method
//   that's
//   called by itself when we create
//   object for that class


// argument => we pass value when function is called
// parameter => we receive value in the function definition


// inheritance => we can access parent
//   properties in child class

class Parent{

```

```
    age = 20;
    greet(){
        console.log("Greet method from parent class");
    }
}

class Child extends Parent{
    display(){
        console.log("display method from child class");
    }
}

let child = new Child();
child.display();

console.log(child.age);
child.greet();

// method overriding
// same method name with different implementation
// in different subclass

class Animal{
    sound(){
        console.log("sound.....");
    }
}

class Dog extends Animal{
    sound(){
        super.sound();
        console.log("Barking...");
    }
}

let dog = new Dog();

dog.sound();
```

```
// 1gb => 1024 MB

// 1 mb => 1024 kb(kilo bytes)

// 1 byte => 8 bit

// 1 bit => 0 or 1
</script>
</body>
</html>
```